

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №6

Работа с коллекциями

по дисциплине

«Информационные технологии и программирование»

Выполнил: студент гр. БВТ2403

Толибов Р.Ш.

Руководитель:

Москва, 2025 г.

Цель работы: изучить основные интерфейсы и классы коллекций в языке программирования Java, освоить использование обобщённых типов (дженериков), итераторов, а также получить практические навыки применения коллекций для решения прикладных задач обработки данных.

Ход работы:

Коллекции в Java представляют собой набор интерфейсов и классов, предназначенных для хранения и управления группами объектов. Они позволяют эффективно выполнять операции добавления, удаления, поиска и сортировки элементов. Базовым интерфейсом является Collection, от которого наследуются основные типы коллекций.

Интерфейс List представляет упорядоченную коллекцию, допускающую хранение дубликатов. Наиболее распространённые реализации — ArrayList и LinkedList. ArrayList основан на массиве и обеспечивает быстрый доступ к элементам по индексу, но менее эффективен при вставке и удалении элементов в середине списка. LinkedList реализован в виде связного списка, что позволяет быстрее выполнять операции вставки и удаления, но замедляет доступ по индексу.

Интерфейс Set предназначен для хранения уникальных элементов, а интерфейс Map используется для хранения пар «ключ–значение». Очереди представлены интерфейсами Queue и Deque, которые реализуют различные способы организации доступа к элементам.

Дженерики в Java позволяют создавать параметризованные классы и методы, работающие с различными типами данных. Это повышает типобезопасность программ и уменьшает количество ошибок времени выполнения. Однако при компиляции используется механизм стирания типов, в результате которого информация о параметрах типов удаляется, а обобщённый код преобразуется в необобщённый.

Итераторы используются для последовательного обхода элементов коллекций. Они позволяют безопасно удалять элементы во время обхода без нарушения целостности коллекции.

Задание 1. Поиск топ-10 наиболее часто встречающихся слов.

Была разработана программа, считывающая текстовый файл и определяющая десять наиболее часто встречающихся слов. Для хранения слов и количества их повторений использовалась коллекция Map, где ключом являлось слово, а значением — число его вхождений.

```

import java.io.*;
import java.util.*;

public class TopWords {
    public static void main(String[] args) {
        String fileName = "text.txt";
        File file = new File(fileName);
        Scanner scanner = null;

        try {
            scanner = new Scanner(file);
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }

        Map<String, Integer> wordCount = new HashMap<>();
        while (scanner.hasNext()) {
            String word = scanner.next().toLowerCase().replaceAll("[^a-zA-Я0-9]", "");
            if (word.isEmpty()) continue;
            wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
        }
        scanner.close();

        List<Map.Entry<String, Integer>> list = new
ArrayList<>(wordCount.entrySet());

        Collections.sort(list, new Comparator<Map.Entry<String, Integer>>() {
            @Override
            public int compare(Map.Entry<String, Integer> o1,
Map.Entry<String, Integer> o2) {
                return o2.getValue().compareTo(o1.getValue());
            }
        });

        int limit = Math.min(10, list.size());
        System.out.println("Топ " + limit + " слов:");

        for (int i = 0; i < limit; i++) {
            Map.Entry<String, Integer> entry = list.get(i);
            System.out.println((i + 1) + ") " + entry.getKey() + " = " +
entry.getValue());
        }
    }
}

```

Задание 2. Реализация обобщённого класса Stack<T>

Был реализован обобщённый класс Stack<T>, представляющий стек на основе массива. Класс поддерживает следующие операции:

- push — добавление элемента в стек;
- pop — удаление и возврат верхнего элемента стека;
- peek — получение верхнего элемента без его удаления.

Для хранения элементов использовался массив типа Object, приведённый к типу T, что обусловлено ограничениями, связанными со стиранием типов.

Реализация позволила использовать стек с любыми типами данных, включая Integer.

```
public class Stack<T> {
    private T[] data;
    private int size;
    @SuppressWarnings("unchecked")
    public Stack(int capacity) {
        data = (T[]) new Object[capacity];
        size = 0;
    }
    public void push(T element) {
        data[size] = element;
        size++;
    }
    public T pop() {
        size--;
        T element = data[size];
        data[size] = null;
        return element;
    }
    public T peek() {
        return data[size - 1];
    }
}

public static void main(String[] args) {
    Stack<Integer> stack = new Stack<>(10);
    stack.push(1);
    stack.push(2);
    stack.push(3);
    System.out.println("Удален последний элемент: " + stack.pop());
    System.out.println("Последний элемент в стеке: " + stack.peek());
}
```

Задание 3. Учёт продаж в магазине

Была разработана программа для учёта продаж в магазине. Программа позволяет:

- добавлять проданные товары в коллекцию;
- выводить список всех проданных товаров;
- вычислять общую сумму продаж;
- определять наиболее популярный товар.

В процессе выполнения задания была использована коллекция `LinkedList` для хранения информации о проданных товарах. Анализ данных осуществлялся с помощью обхода коллекции и подсчёта количества повторений товаров.

```
import java.util.*;

public class SalesManager {
    private static class Sale {
        private String name;
        private double price;

        public Sale(String name, double price) {
            this.name = name;
            this.price = price;
        }

        public String getName() {
            return name;
        }

        public double getPrice() {
            return price;
        }
    }

    @Override
    public String toString() {
        return name + " - " + price + " руб.";
    }
}

private LinkedList<Sale> sales = new LinkedList<>();
public void addSale(String name, double price) {
    sales.add(new Sale(name, price));
}

public void printSales() {
    if (sales.isEmpty()) {
        System.out.println("Продаж пока нет.");
        return;
    }
    System.out.println("Список проданных товаров: ");
    for (Sale s : sales) {
        System.out.println(s);
    }
}
```

```

    }

    public double totalSum() {
        double sum = 0;
        for (Sale s : sales) {
            sum += s.getPrice();
        }
        return sum;
    }

    public String mostPopularProduct() {
        if (sales.isEmpty()) {
            return "Нет данных";
        }
        Map<String, Integer> countMap = new HashMap<>();
        for (Sale s : sales) {
            countMap.put(s.getName(),
countMap.getDefault(s.getName(), 0) + 1);
        }
        String popular = null;
        int maxCount = 0;
        for (Map.Entry<String, Integer> entry : countMap.entrySet()) {
            if (entry.getValue() > maxCount) {
                popular = entry.getKey();
                maxCount = entry.getValue();
            }
        }
        return popular;
    }

    public static void main(String[] args) {
        SalesManager manager = new SalesManager();

        manager.addSale("Хлеб", 39.99);
        manager.addSale("Молоко", 69.99);
        manager.addSale("Хлеб", 89.99);
        manager.addSale("Яблоко", 39.99);

        manager.printSales();

        System.out.println("\nОбщая сумма продаж: " +
manager.totalSum() + " руб.");
        System.out.println("Самый популярный товар: " +
manager.mostPopularProduct());
    }
}

```

Вывод: в результате выполнения лабораторной работы были изучены основные интерфейсы и классы коллекций Java, механизм обобщённых типов и особенности стирания типов. Коллекции показали свою эффективность при решении задач хранения, анализа и обработки данных. Полученные знания могут быть использованы при разработке реальных приложений, работающих с большими объёмами информации.

<https://github.com/TRuslan666/ITiP/tree/main/лаб6>